

?# Relatório TDD - Passo a Passo

Objetivo

Este relatório explica, de forma direta, como o ciclo de TDD foi aplicado no projeto 'micro-erp-auto' para testes de 'ClienteService' e 'AuthService'.

O que foi criado

- Projeto de testes 'MicroERP.Tests'.
- Testes com 'xUnit'.
- Mocks com 'Moq' para dependências de repositório.
- Uso de 'EF Core InMemory' para cenários de autenticação sem banco real ativo.

Estrutura principal

- 'MicroERP.Tests/Services/ClienteServiceTests.cs'
- 'MicroERP.Tests/Services/AuthServiceTests.cs'
- 'MicroERP.Tests/MicroERP.Tests.csproj'

Passo a passo do ciclo TDD usado

1. Escolher um comportamento específico já existente no sistema.
2. Escrever o teste primeiro, descrevendo o resultado esperado.
3. Rodar os testes e confirmar falha inicial quando aplicável (fase RED).
4. Ajustar apenas o necessário no código de suporte de teste (fase GREEN).
5. Refatorar os testes para manter clareza e reutilização (fase REFACTOR).
6. Rodar toda a suite novamente para garantir estabilidade.

Exemplos aplicados

ClienteService

- 'CreateAsync':
 - deve falhar com exceção quando CPF já existe.
 - deve criar cliente com CPF normalizado quando CPF é novo.
- 'UpdateAsync':
 - retorna 'null' quando cliente não existe.
 - lança exceção quando CPF novo já pertence a outro cliente.
 - atualiza e persiste quando CPF está disponível.
- 'DeleteAsync':
 - retorna 'true' e marca exclusão lógica quando cliente existe.
 - retorna 'false' quando cliente não é encontrado.

AuthService

- 'RegisterAsync':
 - deve falhar quando email já existe.
 - deve persistir usuário com senha em hash.
- 'LoginAsync':
 - retorna token quando credenciais estão corretas.
 - retorna 'null' quando senha está incorreta.

Resultado obtido

- Suite de testes executada com sucesso.
- Total atual validado: **11 testes aprovados**.
- Regras principais da API preservadas para este ciclo.

Como executar

Na raiz do repositório:

```
““bash
dotnet test MicroERP.sln
““
```